

Reasoning LLMs

Module 1 — What they are, why now

Lucas Soares · O'Reilly Live Training

What do we mean by "reasoning"?

Recall — look something up:

"What's the course code for Stanford's Transformers class?" → CME 295

Reasoning — work it out:

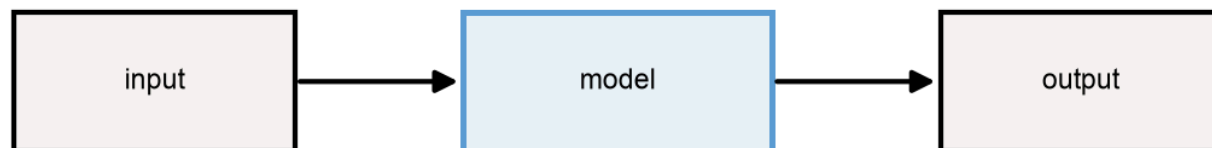
"A bear was born in 2020. How old in 2025?" → $2025 - 2020 = 5$

Reasoning = solving a problem through a **multi-step process**, not a single lookup.

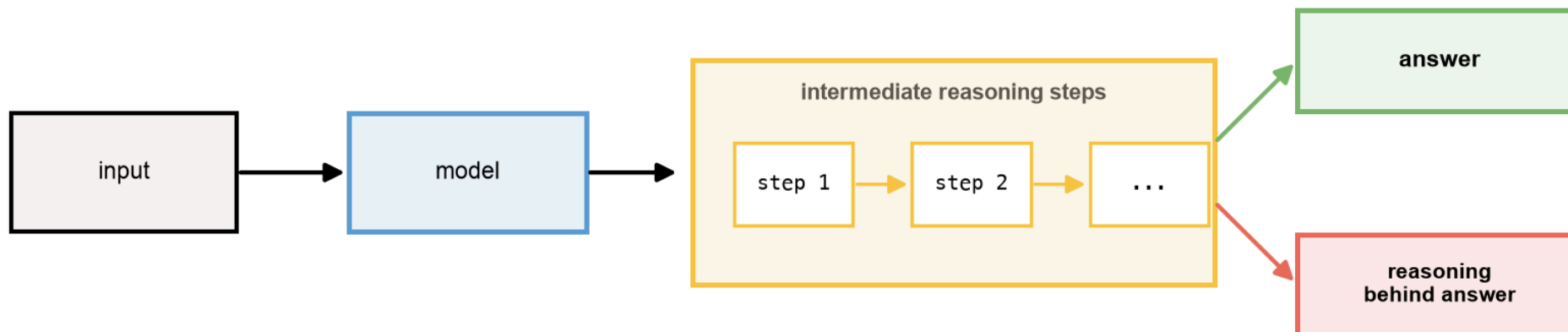
What is a reasoning LLM?

Reasoning LLM = reasoning chain + answer

Traditional LLM



Reasoning LLM



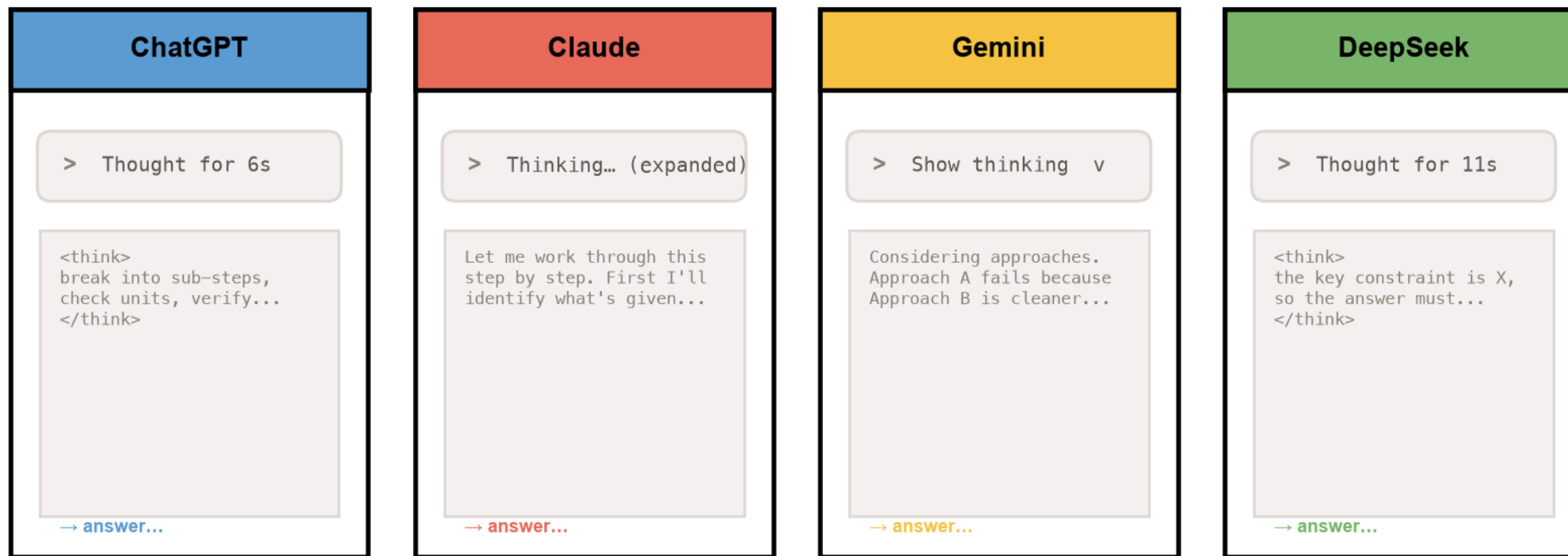
Output = reasoning chain + answer.

The output contract changes: **Output = reasoning chain + answer**

You've already seen these everywhere

Thinking shows up everywhere

illustrative mockups — not real screenshots



A catch: you pay for the thoughts

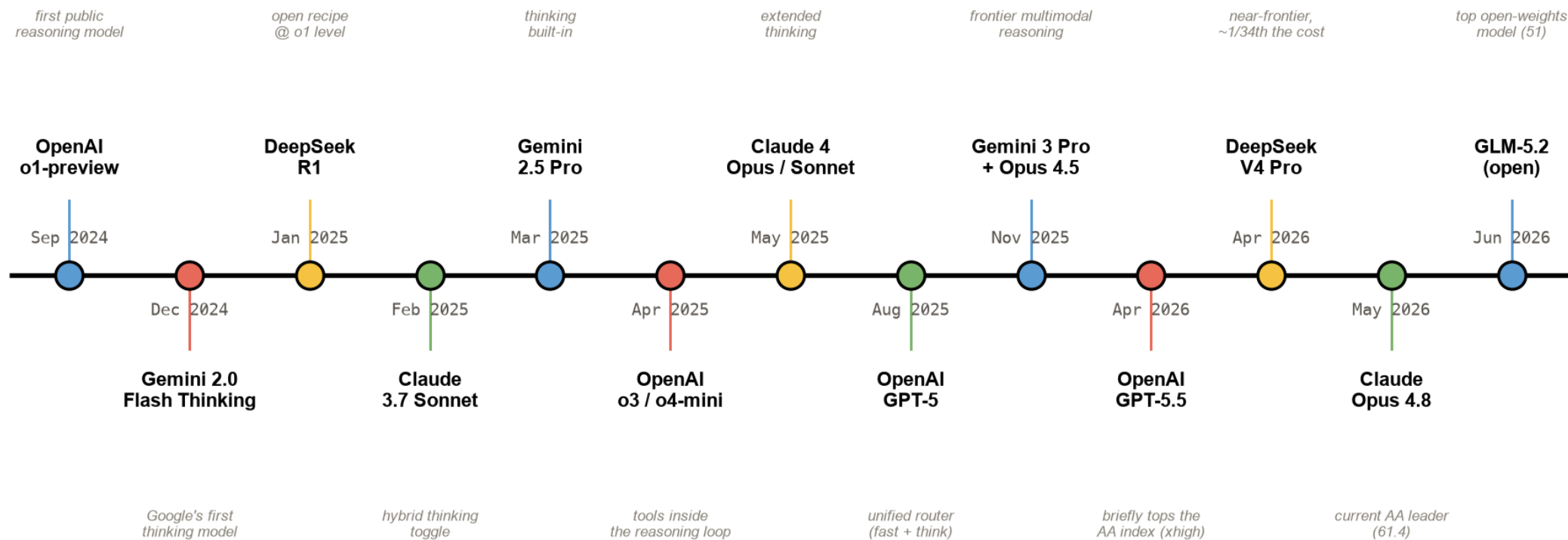
Reasoning tokens are **billed as output tokens** — on every major API.

- UIs show a *summarized* thought, not the raw chain
(barely-readable, and exposed chains could train rivals)
- So there's a real incentive to get **the most reasoning per token**

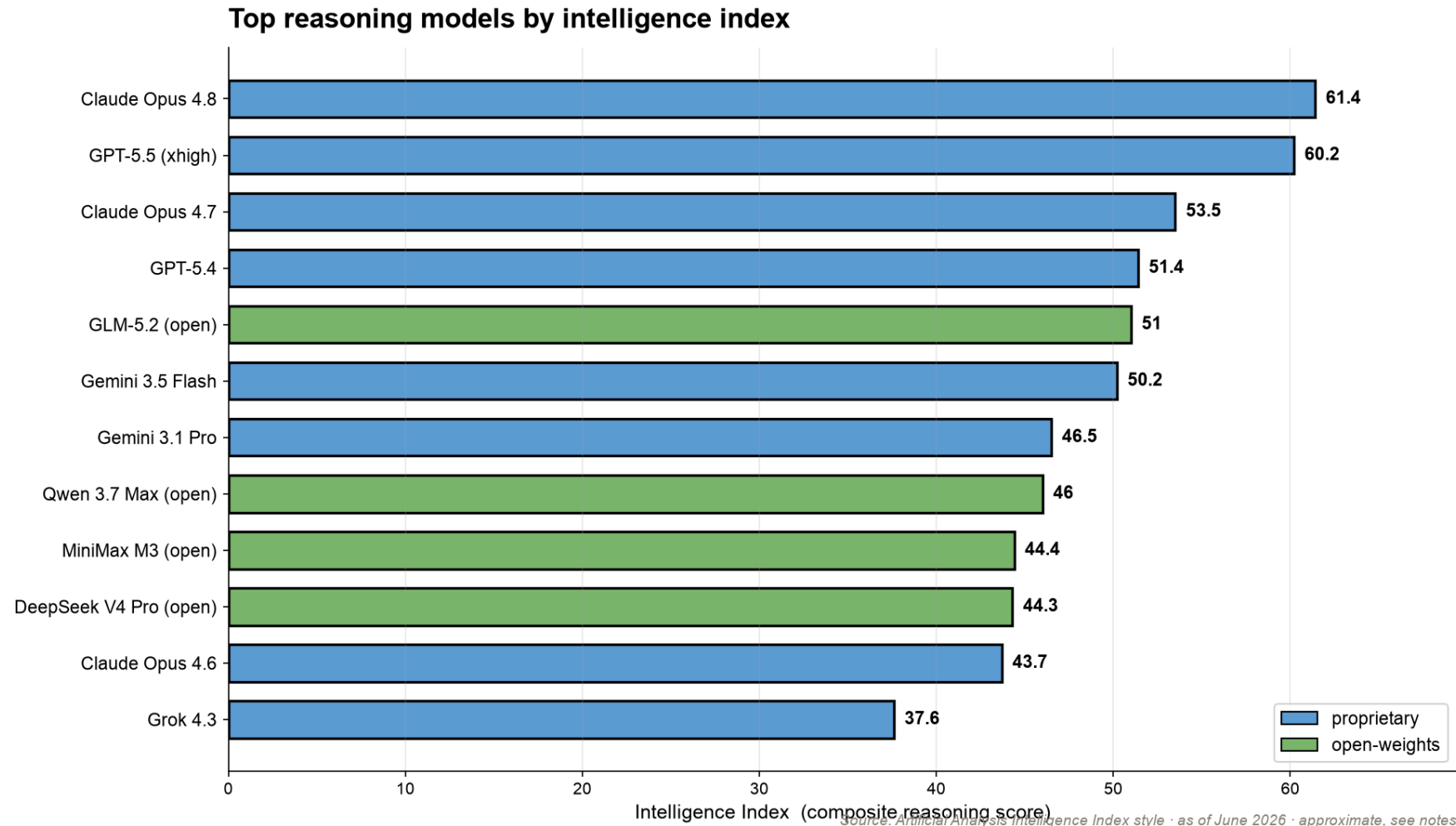
More on controlling the thinking budget later.

How we got here

The reasoning-model wave · Sep 2024 → mid 2026

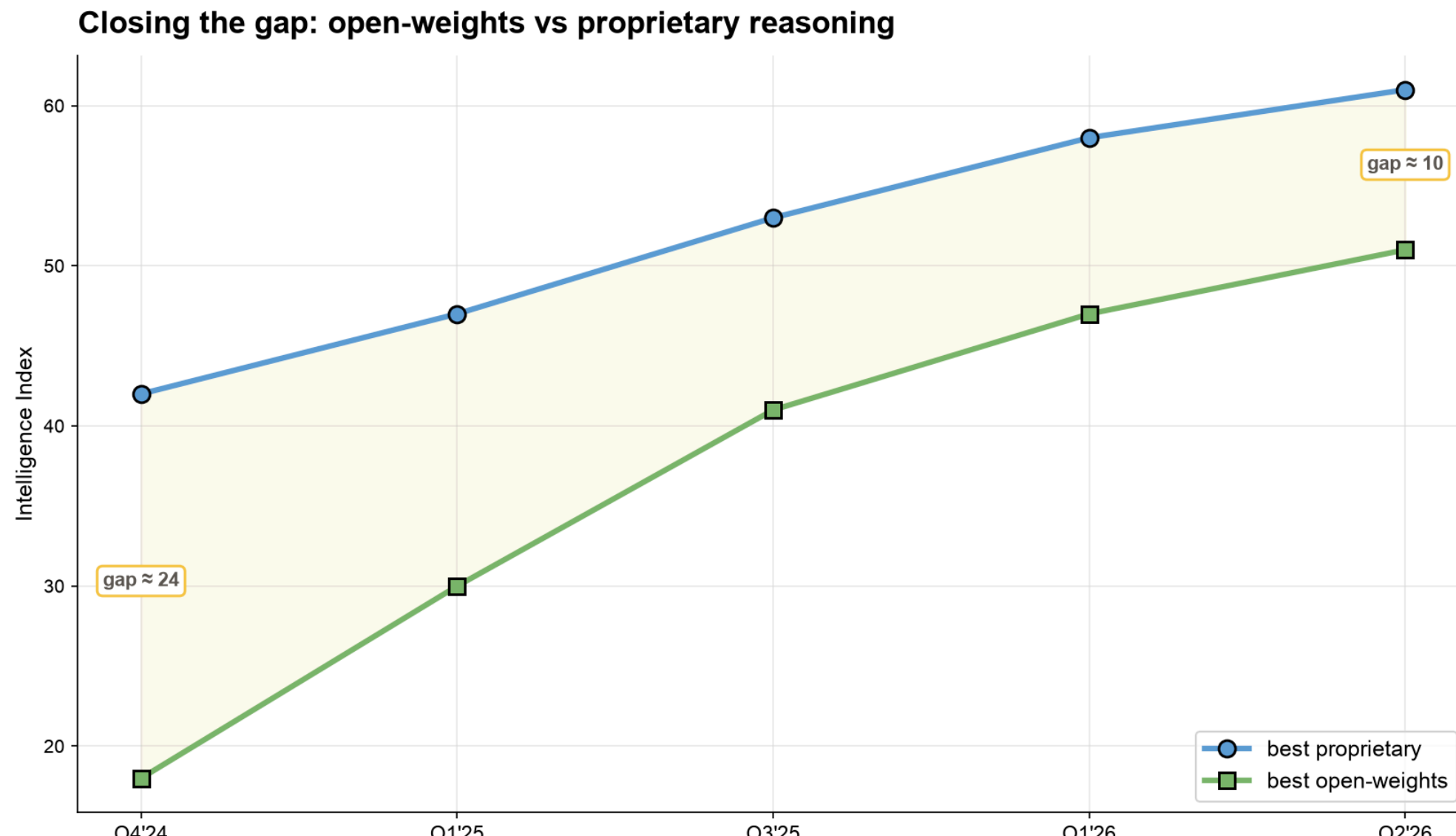


Where the models rank today



Demand concentrates on a handful of top labs. (*Artificial Analysis, ~June 2026.*)

Closing the gap



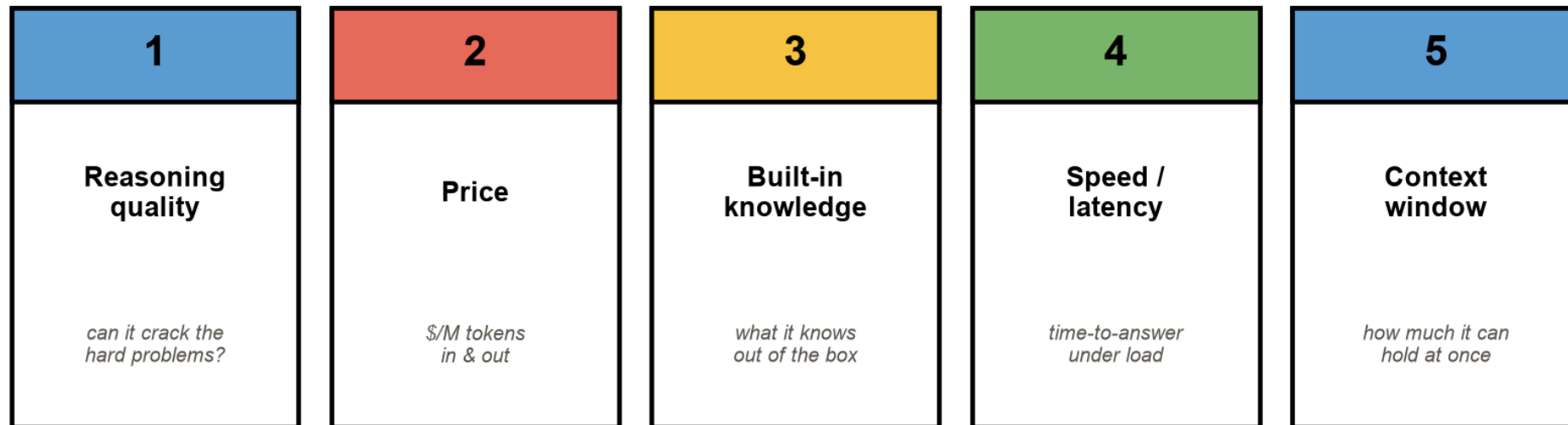
Why reach for a reasoning model?

It earns its extra tokens when the task is a **derivation**, not a lookup:

- Decomposes hard, multi-constraint problems
- Hallucinates less on problems it can *check itself* on
- Gives you the **reasoning**, not just the verdict

What actually matters when you pick one

Choosing a reasoning model — 5 criteria



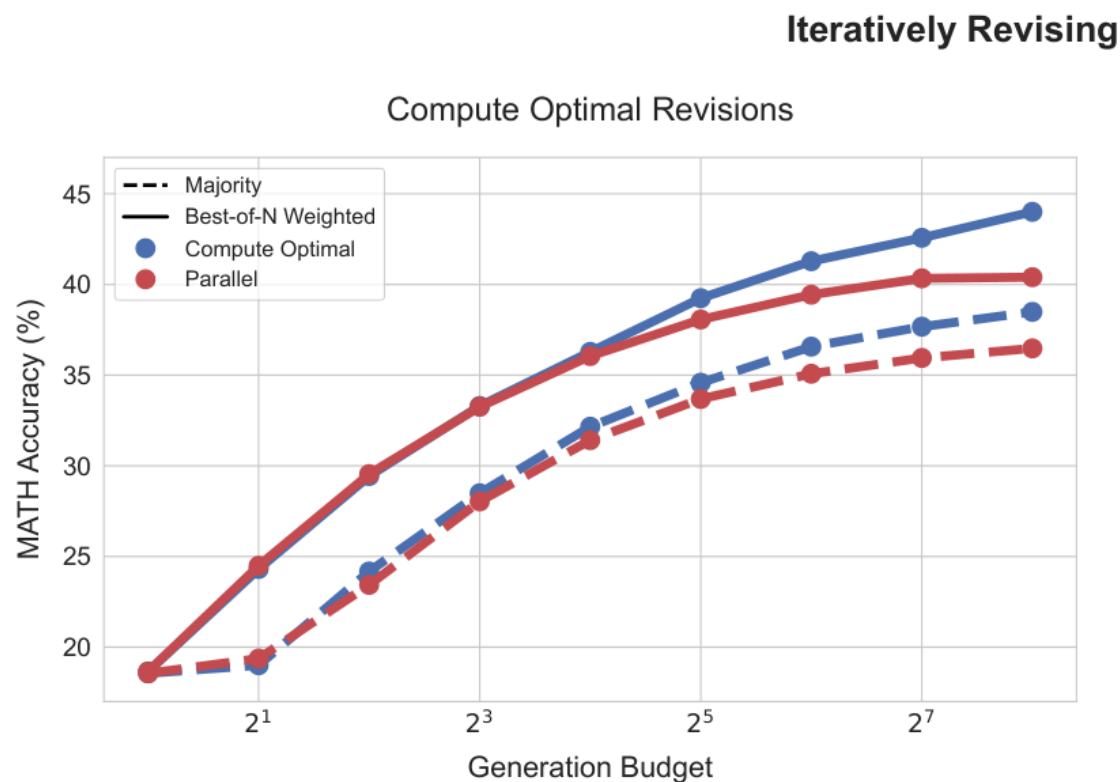
Reasoning quality · price · built-in knowledge · speed · context window.

Price is the other axis

Model (~June 2026)	\$/M out	
GPT-5.5	~\$30	frontier
Claude Opus 4.8	~\$25	frontier
Gemini 3.1 Pro	~\$12	
DeepSeek V4 Pro	~\$0.87	near-frontier, ~ 34× cheaper

The expensive model isn't always the right call. (*Approx; we test this in notebook 08.*)

Why does thinking longer help?



Every generated token is another forward pass — reasoning tokens **buy compute** before the model commits.

The simplest version of the trick

Chain-of-thought prompting elicits reasoning

recreation — after Wei et al., 2022 (arXiv:2201.11903)

Standard prompting	Chain-of-thought prompting
Q: Roger has 5 tennis balls. He buys 2 more cans of 3 balls each. How many does he have now? A: The answer is 11. (no working shown) X	Q: Roger has 5 tennis balls. He buys 2 more cans of 3 balls each. How many does he have now? A: Roger started with 5 balls. 2 cans of 3 balls each is 6 balls. $5 + 6 = 11$. The answer is 11. /

no reasoning -> wrong

reasoning steps -> right

"Let's think step by step." Chain-of-thought, scaled up, is the core idea. *(after Wei et al., 2022)*

Two reasons a reasoning chain helps

1. **Decomposition** — break a problem the model has never seen into sub-problems it *has*.
2. **More compute** — more tokens before the answer = more computation spent on it.

Extended chains and self-correction ("wait, that's wrong...") **emerge from RL**, not just from prompting.

When to use one — and when not to

When to reach for a reasoning model

GOOD FIT	POOR FIT
+ Complex, single-shot problems + Fewer hallucinations needed + Diagnosis-style reasoning + Explanations & step-by-step + Hard problems other LLMs fail	- Speed / latency-critical tasks - Cheap bulk extraction - High-volume summarization - Simple lookups & formatting - Tight per-call cost budgets

Great for hard, single-shot problems. Wasteful for cheap, high-volume extraction.

How to prompt a reasoning model

- Keep the prompt **short and direct** — don't hand-write the chain of thought for it
- Frame it as an **expert generalist**; state the goal, not the steps
- Give it **clear success/failure criteria** for what a good answer looks like
- Tune **effort/budget** to the task instead of always maxing it

You're not prompting a model — you're prompting an agent

Prompt it like an agent, not a model

Goal	What you ultimately want accomplished.
Return format	Exact structure / schema of the output.
Warnings	Edge cases, what to avoid, constraints.
Context	Background, data, prior state.
Tools	web search · code interpreter · retrieval
Actions	Steps it may take to reach the goal.

You're prompting an agent now.

The effort knobs

- **OpenAI:** `reasoning.effort = none / low / medium / high / xhigh`
- **Anthropic:** `thinking.budget_tokens = N` (or adaptive)
- **Google:** thinking config on the Gemini models

Same model, dial the depth to the task.

Limitations (keep them honest)

- 5–50× more tokens → **cost & latency**
- Early/open models: **language mixing**, sensitivity to few-shot
- The visible "thought" is a summary — and **may not be faithful**
(Anthropic: *"reasoning models don't always say what they think"*)

Live demo →

```
notebooks/01_foundations_reasoning_and_cot.ipynb
```

A tiny model learns that **a scratchpad beats answering directly** — the whole idea, measured, on your laptop.

Recap

- Reasoning $\neq$ retrieval — **output = reasoning + answer**
- You can see it (the "Thinking..." tag) and you *pay* for it
- Trade tokens for accuracy — only when the task needs it
- Next: how DeepSeek **trained** one, then we **rebuild** it in code

DeepSeek R1

Module 2 — Reading the paper

arxiv.org/pdf/2501.12948

Why this paper matters

Open weights + an open recipe for o1-level reasoning.

How do you train a model to *think*?

You don't write the reasoning for it. You **incentivize** it:

- Show a few examples of **thinking through** a problem (cold-start)
- Then **reward** the behavior you want with RL:
 - did it get the answer **right**? (accuracy)
 - did it use the **<think>** markers? (format)
 - did it **stay in one language**? (consistency)

No human writes the chains. The model discovers them.

The trick: verifiable rewards

For math and code you can **check the answer automatically** — no learned reward model, no human rater in the loop.

- Math → compare to the parsed ground-truth number
- Code → run the **unit tests**

A cheap, unhackable signal is what makes pure-RL reasoning possible.

R1-Zero vs R1

- **R1-Zero:** pure RL on a base model, no SFT. Works — but ugly outputs.
- **R1:** add a small cold-start SFT + a second SFT pass. Same skill, readable.

The "aha moment"

Table 2 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\dots$

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

$\dots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\dots$

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: $\dots$

$\dots$

Accuracy curve

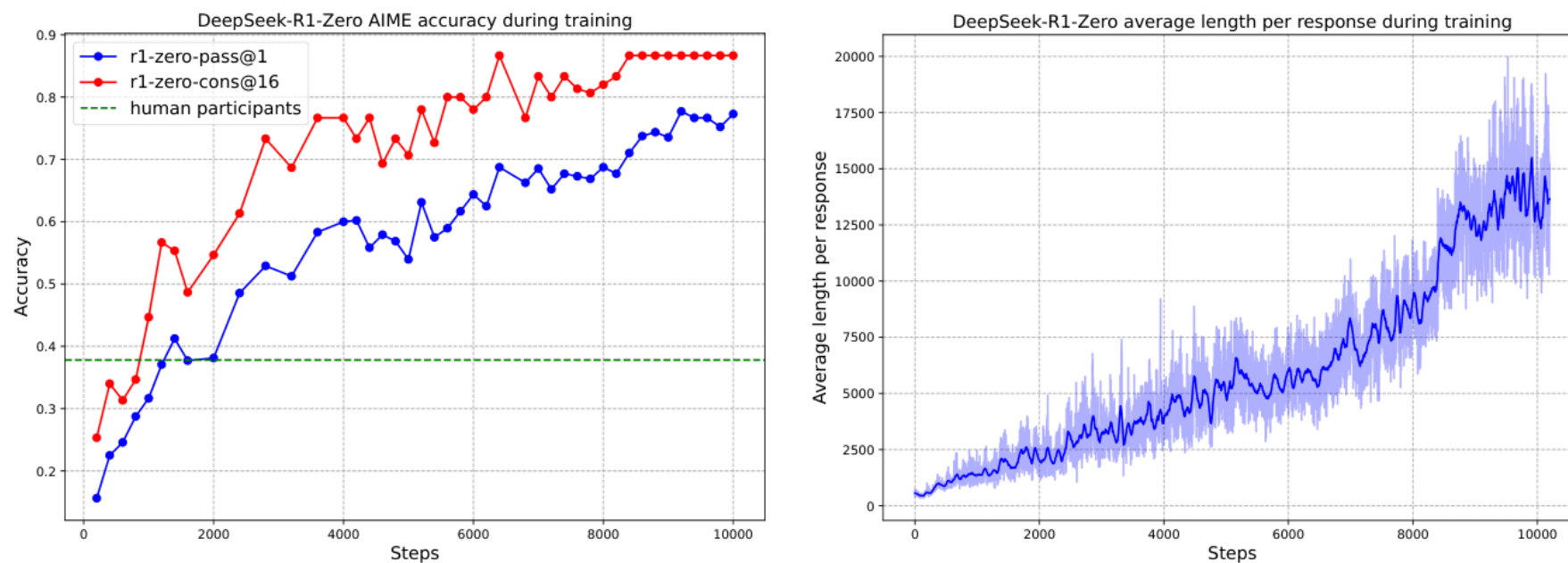


Figure 1 | (a) AIME accuracy of DeepSeek-R1-Zero during training. AIME takes a mathematical problem as input and a number as output, illustrated in Table 32. Pass@1 and Cons@16 are described in Supplementary D.1. The baseline is the average score achieved by human participants in the AIME competition. (b) The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time. Note that a training step refers to a single policy update operation.

The 5-stage recipe

The R1 recipe — 5 stages



GRPO — the 30-second intuition

1. Ask the model the **same question G times** (e.g. $G = 16$)
2. **Score** each answer with simple rules (right? well-formatted?)
3. Each answer's **advantage** = how much better than the group average
4. Nudge the policy $\uparrow$ on better-than-average, $\downarrow$ on worse-than-average

No value net. No human rater. The **group is the baseline**.

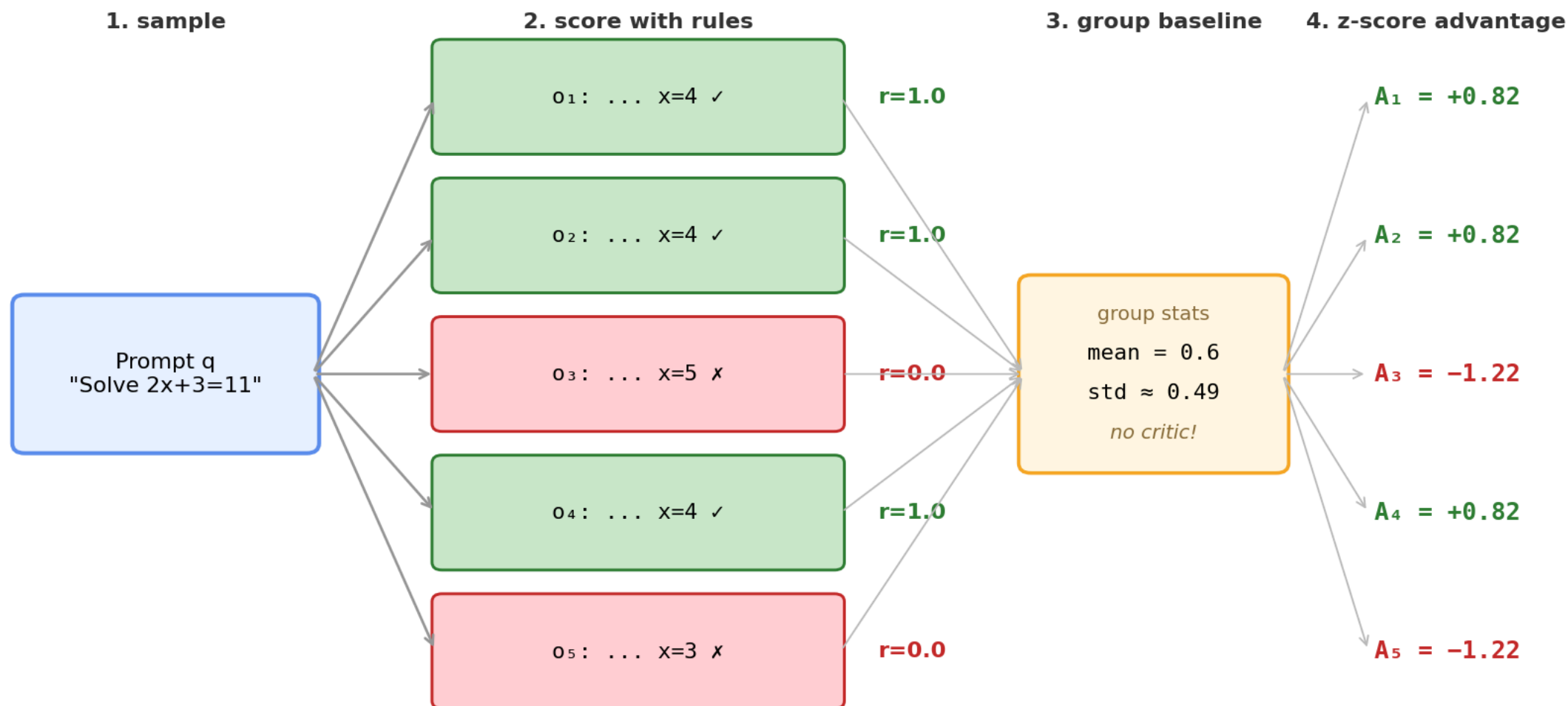
Why no critic?

	PPO (classic RL)	GRPO (DeepSeek)
Baseline	learned value model (extra NN)	mean reward of the group
Networks trained	policy + value	policy only
Memory footprint	~2×	~1×
Reward source	learned reward model (usually)	rule-based (correct? formatted?)

Half the moving parts. No reward hacking from a learned critic.

GRPO in one picture

GRPO in one picture — the group IS the baseline



The formal objective

For each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_{θ} by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

where π_{ref} is a reference policy, ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

Clipped PPO-style update — but the advantage $\hat{A}$ is just the **z-score of the group rewards**. That's the whole trick.

Reward design

- **Accuracy reward** — does the final answer match?
- **Format reward** — did it use `<think>...</think>` ?

No learned reward model. Rule-based only.

Distillation results

Table 15 | Comparison of DeepSeek-R1 distilled models and other comparable models on reasoning-related benchmarks. Numbers in bold denote the performance is statistically significant (t-test with $p < 0.01$).

Model	AIME 2024		MATH	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633

Limitations

Language mixing, sensitivity to few-shot, weaker on general chat.

Next →

We rebuild every stage in code.

```
presentation/03-r1-recipe-five-stages.md
```

The R1 Recipe in Code

Module 3 — Five stages, three notebooks

One toy task — **1-digit addition** — runs through every stage.

We watch **one number** climb the whole way: the task **pass-rate**.

The map: 5 stages → 3 notebooks

Notebook	Stages it covers	The point
01 · Foundations	base model + why a scratchpad helps	intermediate tokens <i>measurably</i> help
02 · The RL core	cold-start SFT → GRPO	teach the format, then reward correctness
03 · Amplify & compress	rejection-SFT → distillation	self-improve, then shrink

Each notebook runs top-to-bottom on a **laptop CPU** in minutes.

Notebook 01 — Foundations

Every reasoning model starts as a plain **next-token predictor**.

Then the key experiment: train two tiny models on addition — one answers **directly**, one gets a **<think>** scratchpad.

```
notebooks/01_foundations_reasoning_and_cot.ipynb
```

The insight that justifies the whole course

The scratchpad model wins.

Intermediate tokens aren't decoration — they're **computation**.

That single result is *why* reasoning models exist.

Notebook 02 — The RL core (1/2): Cold-start SFT

Teach the **format** first: `<think>...</think><answer>...</answer>` .

A handful of examples locks in the shape — answers still mostly wrong.

Pass-rate: low single digits. That's expected.

Notebook 02 — The RL core (2/2): GRPO

Sample the same prompt **G times**, score each by rule, push toward the group's best.

No critic. No value network. No human labels — **the group is the baseline**.

Watch the pass-rate climb.

```
notebooks/02_rl_core_coldstart_sft_and_grpo.ipynb
```

Notebook 03 — Amplify (1/2): Rejection-sampling SFT

Use the RL'd model as a **data factory**: generate → keep only correct → fine-tune.

The pass-rate jumps again with **no new training signal** — just better data.

Notebook 03 — Compress (2/2): Distillation

Pour the skill into a **smaller student** with KL on the logits.

A model **~1/7 the size** keeps most of the pass-rate.

```
notebooks/03_amplify_and_compress_rejection_sft_and_distillation.ipynb
```

The payoff — one metric, the whole recipe

Stage	What it adds	Pass-rate
Cold-start	format	—
GRPO	skill	■
Reject-SFT	stability	■
Distill	efficiency	■ (smaller)

(Exact numbers print live in the notebook — that's the demo.)

This is still how it's done

The same recipe — **GRPO + RL from verifiable rewards** — is how 2026 frontier reasoning models (GPT-5.5, Claude Opus 4.8, Gemini 3.x) are trained.

Scaled up massively. **Not replaced.**

Bonus → picking & proving in practice

- `notebooks/07_picking_a_reasoning_model_for_an_application.ipynb`
a multi-provider bake-off with an LLM judge
- `notebooks/08_reproducibility_cheap_vs_flagship.ipynb`
fix a **seed**, match a flagship with a **cheaper** model, count the cost

The training story, then the **engineering** story.